

Yazılım ve İşletim Sistemleri

Bilgisayar ve Programlamaya Giriş

Yazılım

İşletim sistemi

Kaynak yönetimi

Sistem türleri

Lisanslama

Sercan KÜLCÜ · Giresun Üniversitesi

Öğrenme Çıktıları

Yazılım türleri

Sistem ve uygulama yazılımlarını ayırt etmek.

İşletim sistemi

İşletim sisteminin kullanıcı, uygulama ve donanım arasındaki rolünü açıklamak.

Kaynak yönetimi

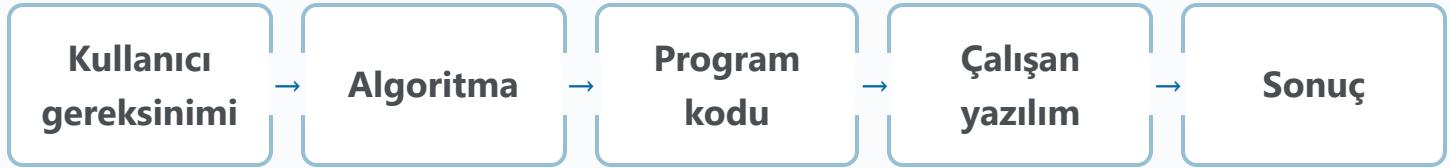
Süreç, bellek, dosya ve aygıt yönetimini yorumlamak.

Sistem seçimi

Farklı işletim sistemi türlerini gereksinimlere göre karşılaştırmak.

Yazılım Nedir?

Yazılım, bir bilgisayarın gerçekleştireceği işlemleri tanımlayan programlar, veriler ve ilgili belgelerin bütünüdür.



Donanım işlemi fiziksel olarak yürütür; yazılım hangi işlemin, ne zaman ve nasıl yapılacağını belirler.

Yazılımın Kalite Özellikleri

İşlevsellik

Beklenen görevleri doğru yerine getirme.

Kullanılabilirlik

Kolay öğrenilme ve etkili kullanım.

Performans

Zaman ve kaynakları verimli kullanma.

Güvenilirlik

Hata durumlarında öngörülebilir davranma.

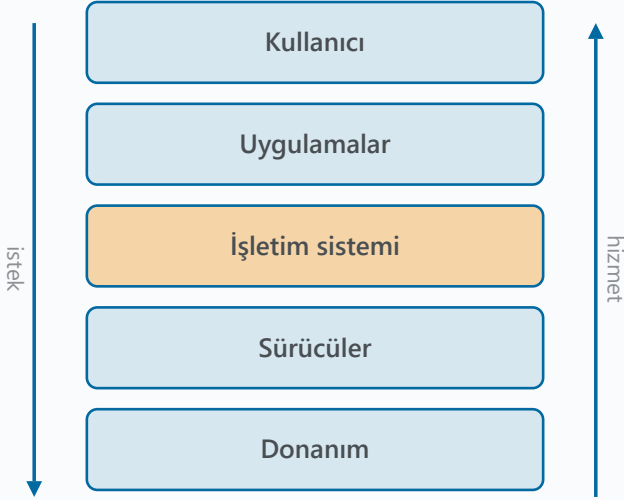
Güvenlik

Yetkisiz erişim ve değişikliklere karşı koruma.

Bakım kolaylığı

Düzeltilme ve geliştirmelerin yönetilebilir olması.

Yazılım ve Donanım İlişkisi



- Kullanıcı yalnızca en üst katmanlarla etkileşir.
- Her katman, altındaki ayrıntıları gizleyerek üst katmana daha kolay hizmetler sunar (soyutlama).
- Aynı uygulama, sürücüler sayesinde farklı donanımlarda çalışabilir.

İstek yukarıdan aşağı iner, sonuç (hizmet) aşağıdan yukarı döner.

Yazılım Türleri

Sistem yazılımı

Donanımı yönetir ve diğer yazılımlara çalışma ortamı sağlar.

Uygulama yazılımı

Kullanıcının belirli işlerini yerine getirir.

Programlama araçları

Kod yazma, çevirme, test ve hata ayıklama işlemlerini destekler.

Ürün yazılımı (firmware)

Bir cihazın temel işlevlerini yöneten gömülü yazılımdır.

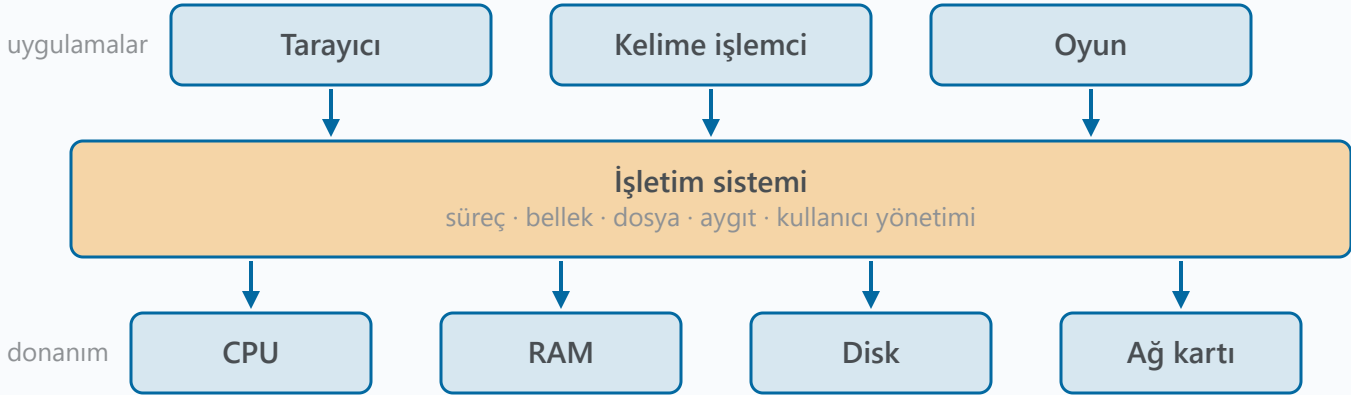


Sistem ve Uygulama Yazılımları

Tür	Örnekler	Temel amaç
Sistem	İşletim sistemi, sürücü, yardımcı araç	Bilgisayarı ve kaynakları yönetmek
Uygulama	Kelime işlemci, tarayıcı, oyun	Kullanıcı görevini gerçekleştirmek
Geliştirme	Derleyici, tümleşik geliştirme ortamı (IDE), hata ayıklayıcı	Yeni yazılım üretmek

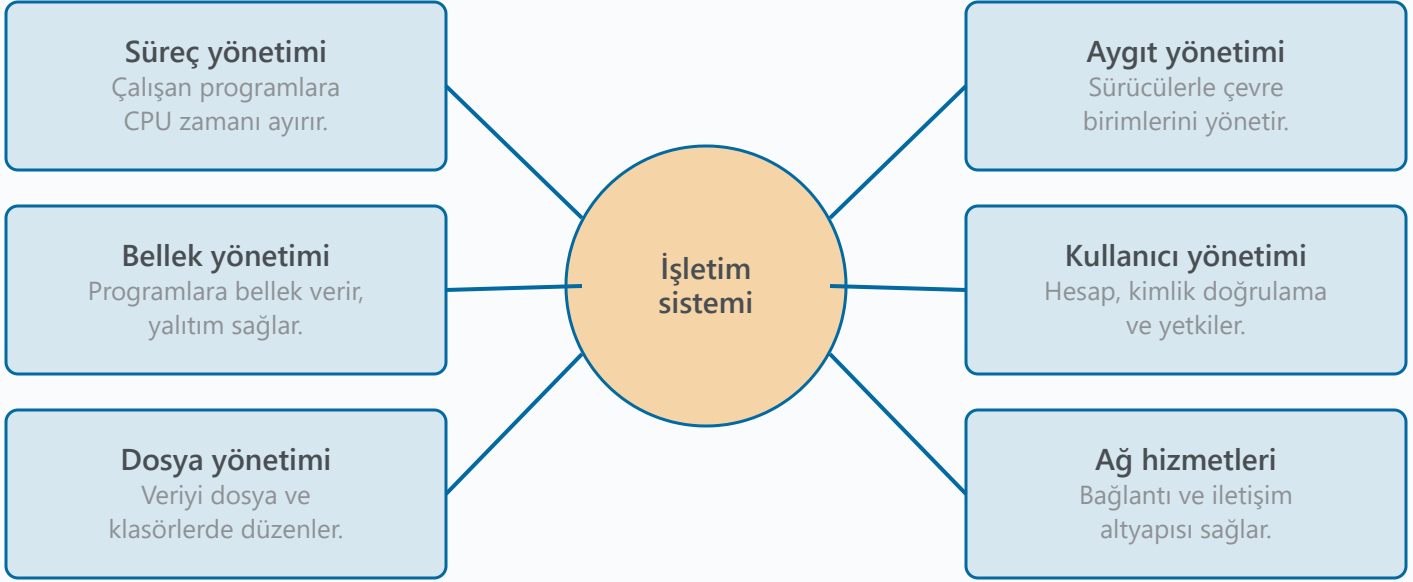
İşletim Sistemi Nedir?

İşletim sistemi, donanım kaynaklarını yöneten ve uygulamalara ortak hizmetler sunan temel sistem yazılımıdır.

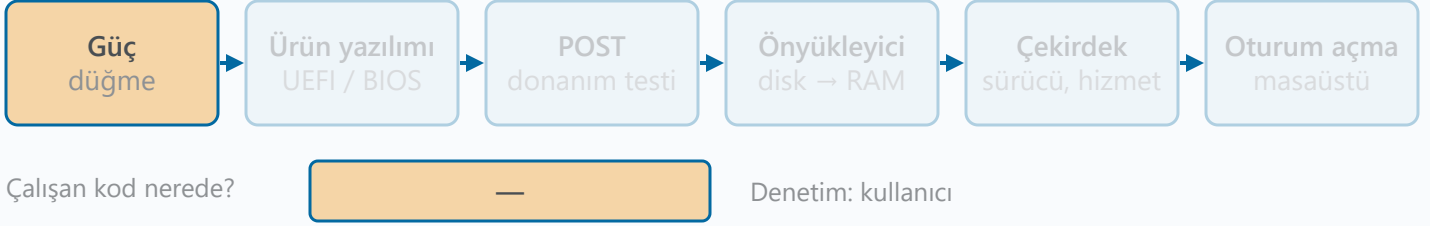


- Donanım ayrıntılarını uygulamalardan soyutlar.
- Birden fazla programın kaynakları güvenli biçimde paylaşmasını sağlar.
- Kullanıcıya grafiksel veya komut satırı arayüzü sunar.

İşletim Sisteminin Görevleri

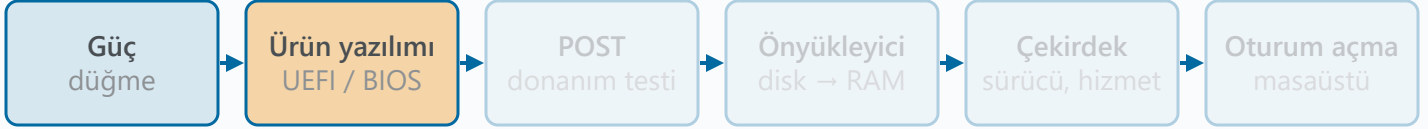


Bilgisayar Nasıl Açılır? Önyükleme Adım Adım



Güç düğmesine basılır. RAM boştur; işletim sistemi henüz diskte bekler.

Bilgisayar Nasıl Açılır? Önyükleme Adım Adım



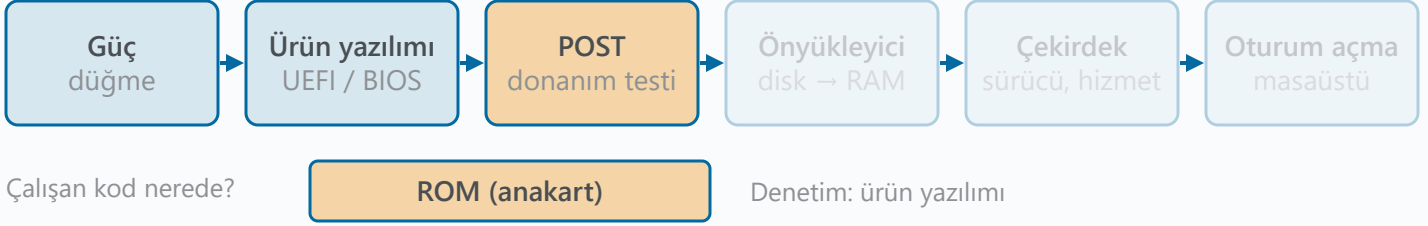
Çalışan kod nerede?

ROM (anakart)

Denetim: ürün yazılımı

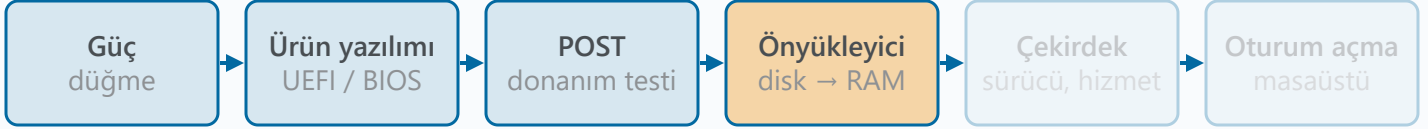
İşlemci, anakarttaki kalıcı bellekte (ROM/flash) duran **ürün yazılımını** (UEFI ya da eski adıyla BIOS) çalıştırmaya başlar.

Bilgisayar Nasıl Açılır? Önyükleme Adım Adım



POST (açılış kendi kendine testi): bellek, klavye, disk gibi temel donanım denetlenir. Hata varsa sesli uyarı ya da ileti verilir.

Bilgisayar Nasıl Açılır? Önyükleme Adım Adım



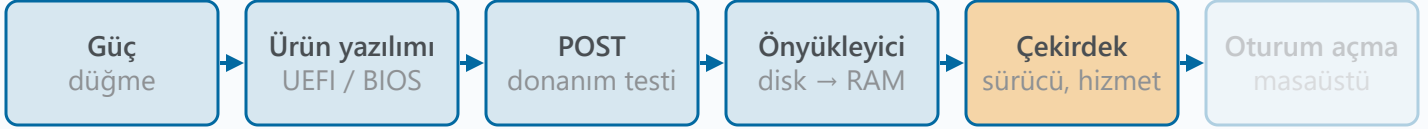
Çalışan kod nerede?

disk → RAM

Denetim: önyükleyici

Ayarlardaki önyükleme sırasına göre disk seçilir; **önyükleyici** (ör. Windows Boot Manager, GRUB) RAM'e yüklenir.

Bilgisayar Nasıl Açılır? Önyükleme Adım Adım



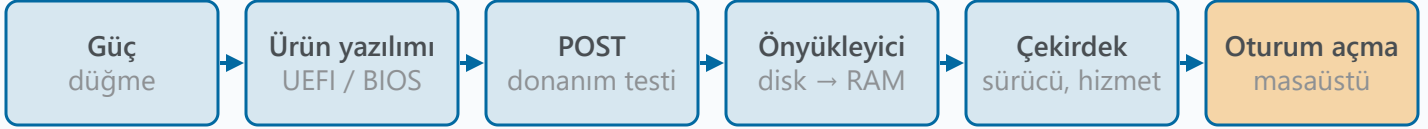
Çalışan kod nerede?

disk → RAM

Denetim: işletim sistemi çekirdeği

Önyükleyici **çekirdeği** RAM'e yükleyip denetimi ona verir. Çekirdek sürücülerini yükler, ilk süreçleri ve sistem hizmetlerini başlatır.

Bilgisayar Nasıl Açılır? Önyükleme Adım Adım



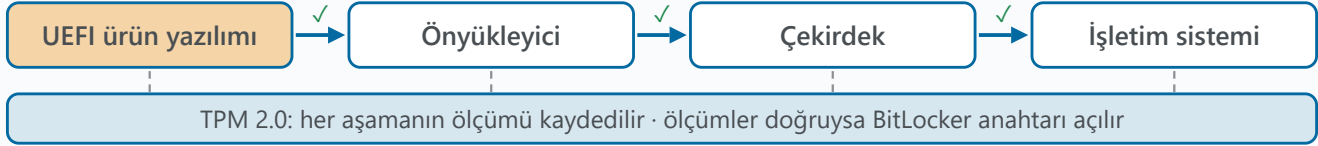
Çalışan kod nerede?

RAM

Denetim: işletim sistemi

Oturum açma ekranı gelir; kimlik doğrulandıktan sonra kullanıcının masaüstü ve başlangıç uygulamaları açılır.

Güvenli Önyükleme ve TPM



UEFI

BIOS'un yerini alan modern ürün yazılımı; büyük disk, hızlı açılış ve imza denetimi sağlar.

Güvenli Önyükleme

Yalnızca güvenilir imzası olan önyükleyici ve çekirdek çalışır; sahte ya da değiştirilmiş önyükleme kodunu engeller.

TPM 2.0

Şifreleme anahtarlarını ve açılış ölçümlerini saklayan güvenlik yongası (anakartta, işlemcide veya ürün yazılımında); BitLocker gibi araçlar kullanır.

Windows 11

Kurulum için UEFI + Güvenli Önyükleme desteği ve TPM 2.0 ister; ayrıntılar Microsoft'un güncel gereksinim sayfasında yer alır.

İki bölümlere düzeni vardır. **MBR:** BIOS ile birlikte, 2 TB'a kadar disk, en çok 4 birincil bölüm. **GPT:** UEFI ile birlikte, 2 TB üzeri disk, çok sayıda bölüm.

Çekirdek (Kernel)

Çekirdek, işletim sisteminin ayrıcalıklı çalışan merkezi bölümüdür. Uygulamalar donanıma doğrudan erişmek yerine sistem çağrılarıyla çekirdekten hizmet ister.

Kullanıcı modu

Metin editörü

Tarayıcı

sistem çağrısı arayüzü

Çekirdek modu (ayrıcalıklı)

Çekirdek

Süreç yön.

Bellek yön.

Dosya sistemi

Aygıt sürücüler

Donanım

CPU

RAM

Disk

Çekirdekteki bir hata tüm sistemi etkileyebilir; bu nedenle erişim denetimi ve yalıtım önemlidir.

Sistem Çağrısı Adım Adım: Dosya Okuma

Kullanıcı modu

Metin editörü

Tarayıcı

sistem çağrısı arayüzü

Çekirdek modu (ayrıcıklı)

Çekirdek

Süreç yön.

Bellek yön.

Dosya sistemi

Aygıt sürücüler

Donanım

CPU

RAM

Disk

Metin editörü **rapor.txt** dosyasını okumak ister. Diske kendisi erişemez; **read** sistem çağrısını yapar.

Sistem Çağrısı Adım Adım: Dosya Okuma

Kullanıcı modu

Metin editörü

Tarayıcı

sistem çağrısı arayüzü

Çekirdek modu (ayrıcılık)

Çekirdek

Süreç yön.

Bellek yön.

Dosya sistemi

Aygıt sürücüler

Donanım

CPU

RAM

Disk

İşlemci **kullanıcı modundan çekirdek moduna** geçer. Artık yalnızca çekirdek kodu çalışır.

Sistem Çağrısı Adım Adım: Dosya Okuma

Kullanıcı modu

Metin editörü

Tarayıcı

sistem çağrısı arayüzü

Çekirdek modu (ayrıcılık)

Çekirdek

Süreç yön.

Bellek yön.

Dosya sistemi

Aygıt sürücüler

Donanım

CPU

RAM

Disk

Dosya sistemi dosyanın diskteki yerini bulur ve kullanıcının **okuma izni** olup olmadığını denetler.

Sistem Çağrısı Adım Adım: Dosya Okuma

Kullanıcı modu

Metin editörü

Tarayıcı

sistem çağrısı arayüzü

Çekirdek modu (ayrıcılık)

Çekirdek

Süreç yön.

Bellek yön.

Dosya sistemi

Aygıt sürücüler

Donanım

CPU

RAM

Disk

Disk sürücüsü diske okuma komutu verir. Disk işi bitince **kesme** (interrupt) ile işlemciye haber verir.

Sistem Çağrısı Adım Adım: Dosya Okuma

Kullanıcı modu

Metin editörü

Tarayıcı

sistem çağrısı arayüzü

Çekirdek modu (ayrıcılık)

Çekirdek

Süreç yön.

Bellek yön.

Dosya sistemi

Aygıt sürücüleri

Donanım

CPU

RAM

Disk

Bellek yönetimi: çekirdek okunan veriyi RAM'de editörün kendi bellek alanına kopyalar.

Sistem Çağrısı Adım Adım: Dosya Okuma

Kullanıcı modu

Metin editörü

Tarayıcı

sistem çağrısı arayüzü

Çekirdek modu (ayrıcıklı)

Çekirdek

Süreç yön.

Bellek yön.

Dosya sistemi

Aygıt sürücüler

Donanım

CPU

RAM

Disk

Çağrı döner, işlemci tekrar **kullanıcı moduna** geçer. Editör veriyi ekranda gösterir. Uygulama donanımı hiç doğrudan görmedi.

Çekirdek Türleri

Tür	Fikir	Örnek
Monolitik çekirdek	Süreç, bellek, dosya sistemi ve sürücüler tek ayrıcalıklı çekirdekte; hızlı ama büyük	Linux (modüller yüklenebilir)
Mikro çekirdek	Çekirdekte yalnızca temel işler; sürücü ve dosya sistemi kullanıcı modunda; yalıtımı güçlü	MINIX 3, QNX
Hibrit çekirdek	Mikro çekirdek tasarımını alıp hız için bazı hizmetleri çekirdekte tutar	Windows NT çekirdeği, macOS/iOS (XNU)

Uygulamalar

Android çatısı ve çalışma zamanı

Linux çekirdeği

Donanım

Android yığını

Linux yalnızca bir çekirdektir; Linux dağıtımı çekirdek + araçlar + masaüstüdür. **Android**, Linux çekirdeğini kullanır ancak üstündeki katmanlar Android'e özgüdür.

Süreç Yönetimi

Program

Diske saklanan komut ve veri bütünüdür.

Süreç

Çalışmakta olan program örneğidir.

İş parçacığı

Süreç içindeki yürütme akışıdır.

Zamanlama

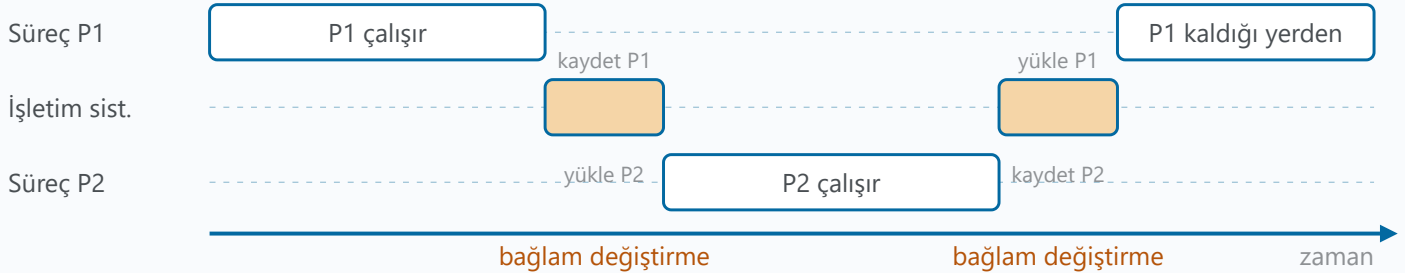
CPU'nun hangi sürece ne zaman verileceğini belirler.

Bağlam değiştirme

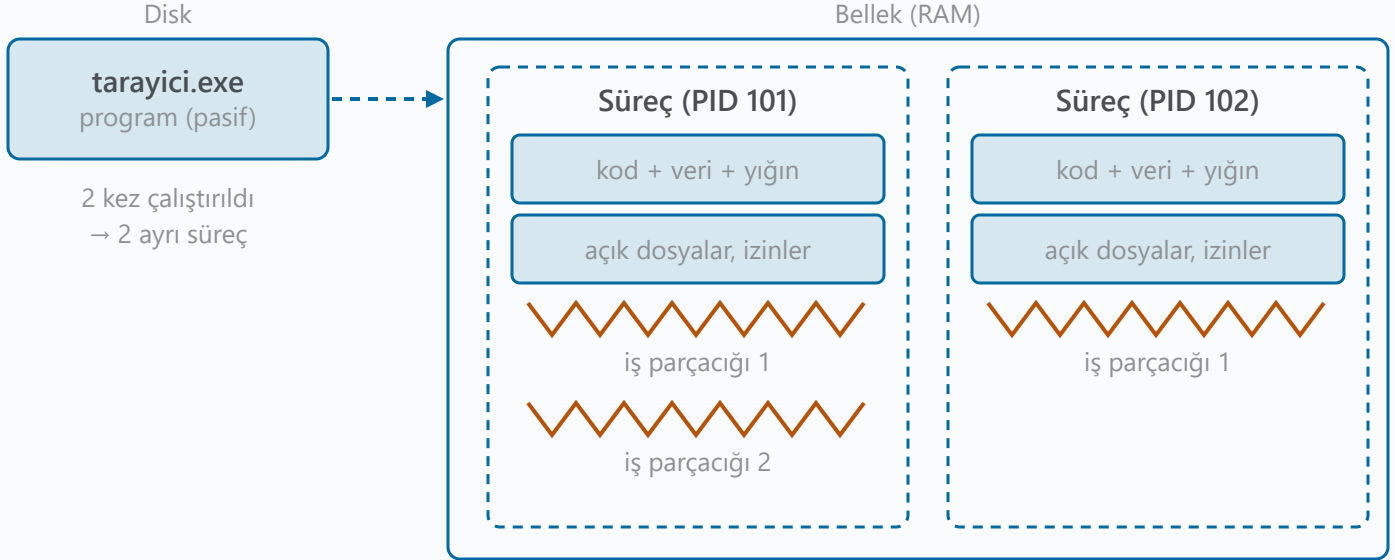
CPU durumunu bir süreçten diğerine aktarır.

Eş zamanlılık

Birden fazla işin ilerlemesini düzenler.



Program, Süreç ve İş Parçacığı



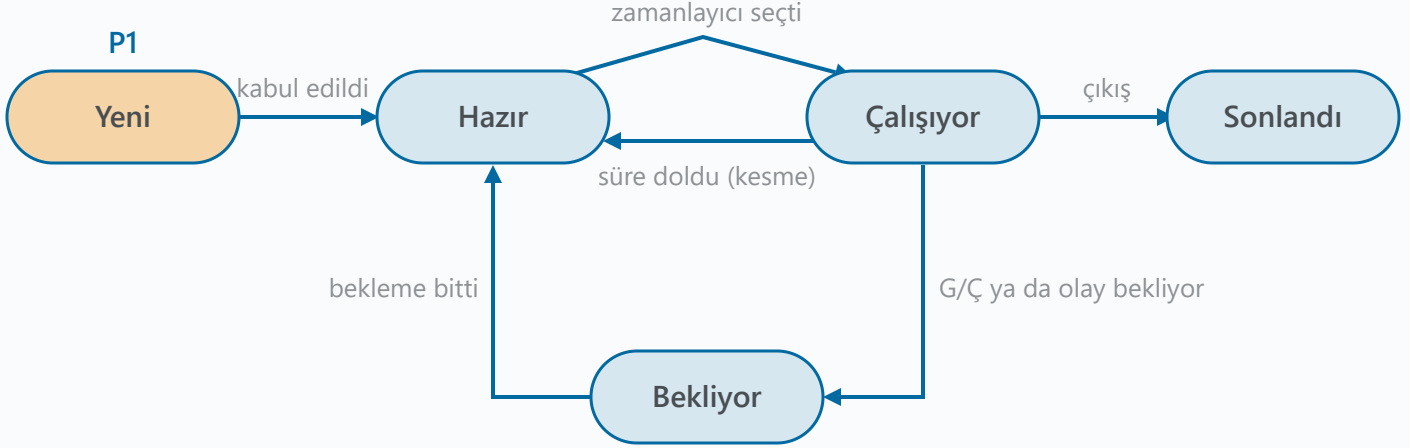
Program diskte duran dosyadır; süreç onun çalışan örneğidir ve bir süreç kimliğiyle (PID) tanınır. Her sürecin ayrı bellek alanı vardır, iş parçacıkları ise süreç içindeki belleği paylaşır.

Süreç Durumları



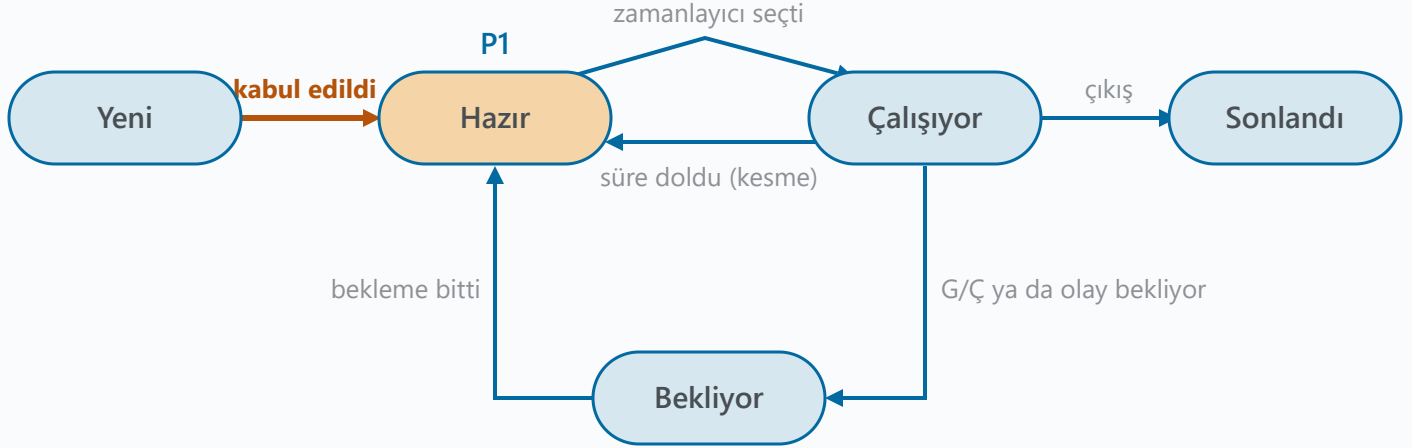
- Tek çekirdekte aynı anda yalnızca **bir** süreç **Çalışıyor** durumundadır; diğer hazır süreçler sırada bekler.
- **Bekliyor** durumundaki süreç CPU'ya verilmez; beklediği olay (disk, klavye, ağ) gerçekleşince yeniden **Hazır** olur.

Süreç Durum Geçişleri Adım Adım



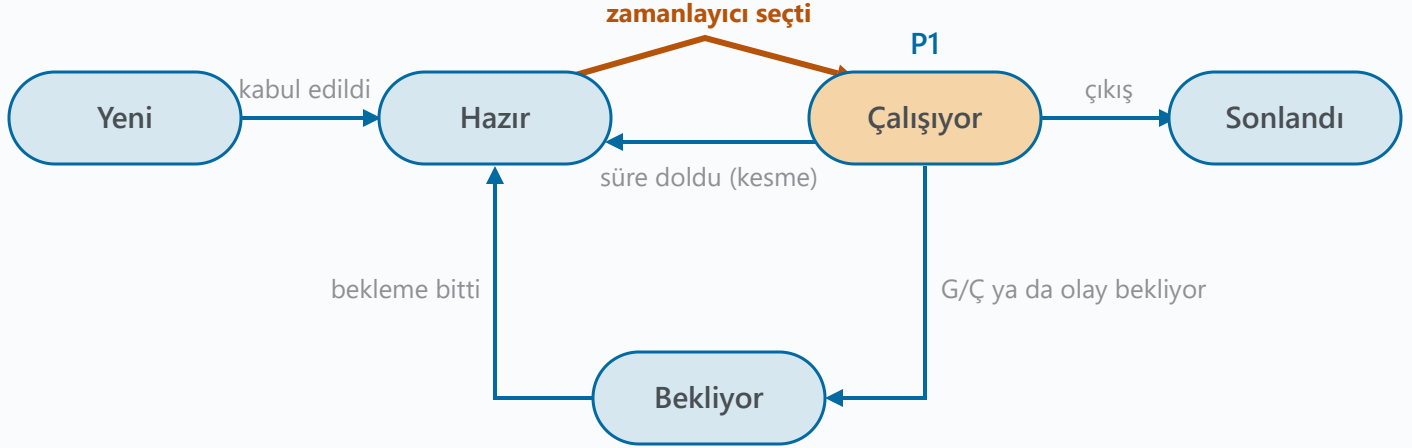
P1 süreci oluşturuluyor (ör. kullanıcı bir programa çift tıkladı): durum **Yeni**.

Süreç Durum Geçişleri Adım Adım



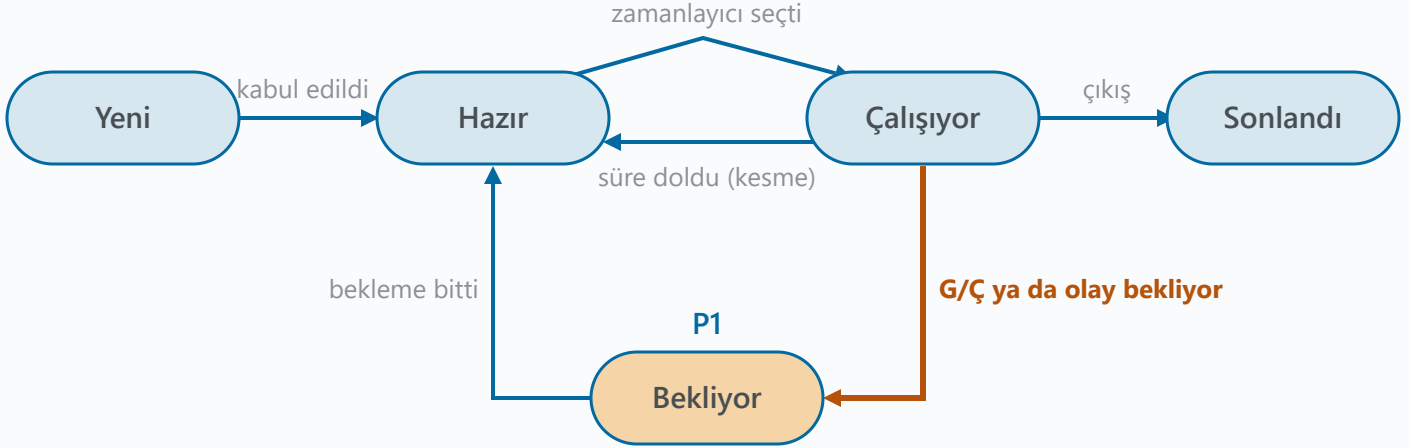
Yeni → **Hazır**: İşletim sistemi süreci **kabul eder**; bellek ayrılır ve süreç hazır kuyruğuna girer.

Süreç Durum Geçişleri Adım Adım



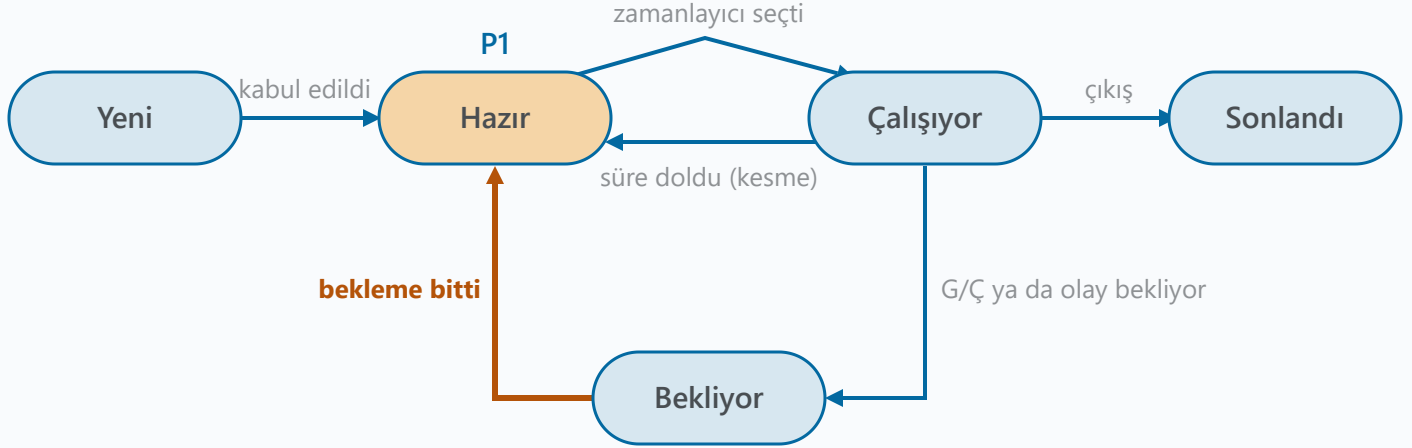
Hazır → **Çalışıyor**: **Zamanlayıcı** P1'i seçer; bağlam yüklenir ve CPU P1'in komutlarını yürütür.

Süreç Durum Geçişleri Adım Adım



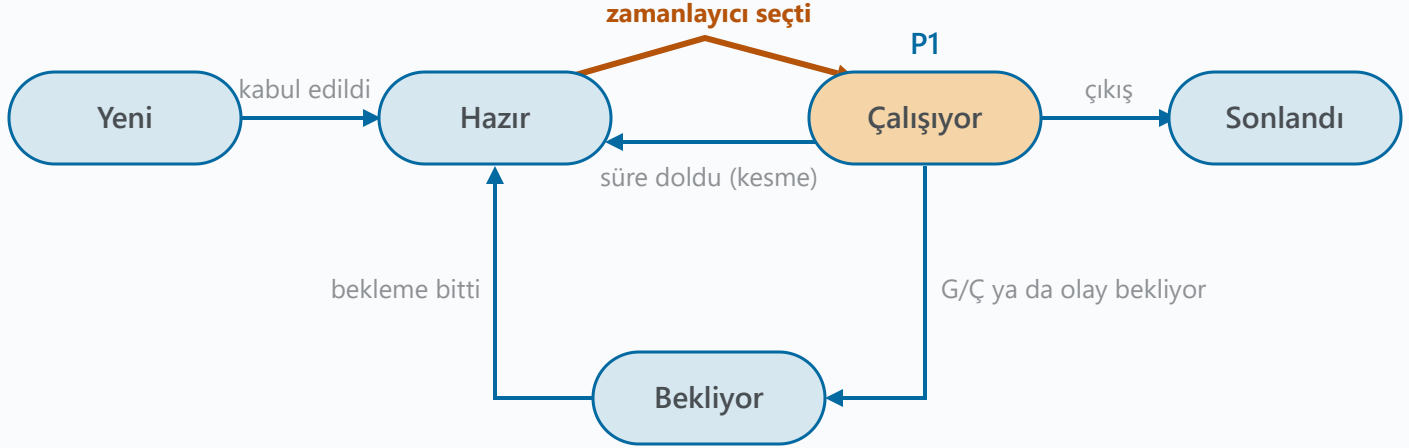
Çalışıyor → **Bekliyor**: P1 diskten veri ister. Veri gelene kadar CPU'yu boşa tutmaması için **Bekliyor** durumuna geçer.

Süreç Durum Geçişleri Adım Adım



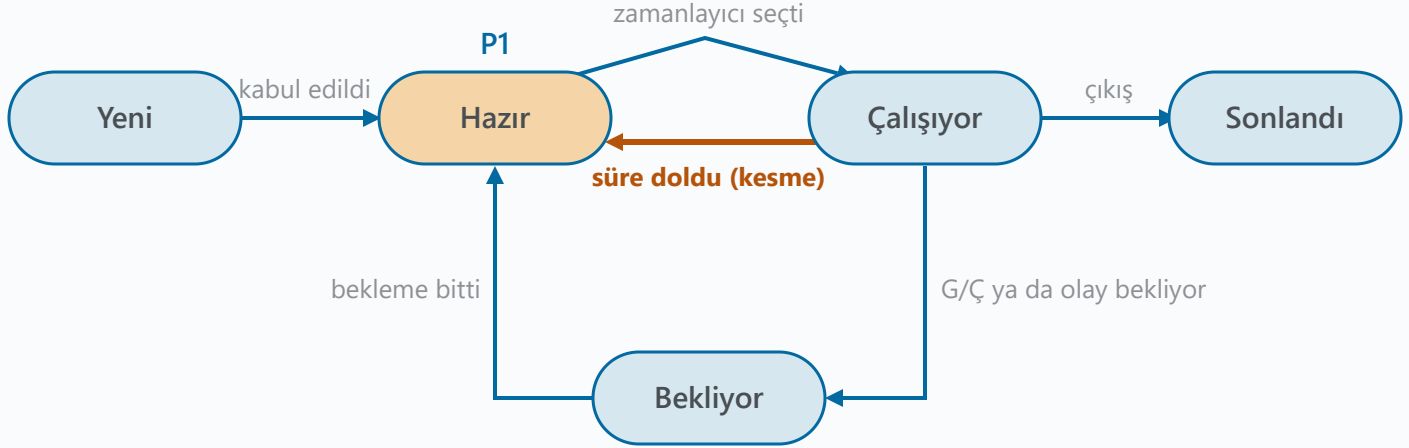
Bekliyor → **Hazır**: Disk işi bitti (kesme geldi). P1 hemen çalışmaz; önce **Hazır** kuyruğuna döner.

Süreç Durum Geçişleri Adım Adım



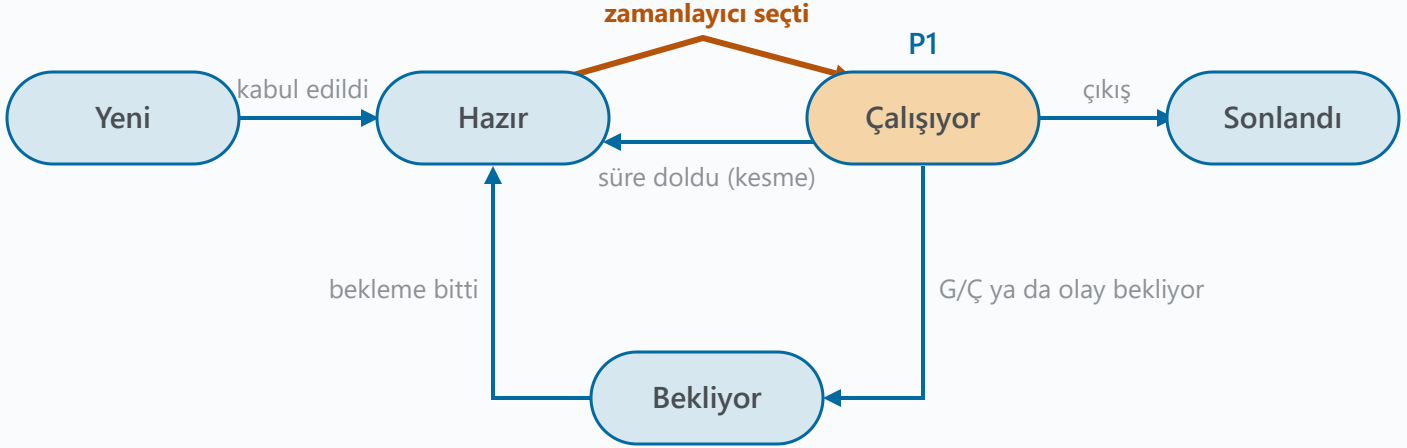
Hazır → **Çalışıyor**: Sırası gelince P1 yeniden seçilir ve kaldığı yerden devam eder.

Süreç Durum Geçişleri Adım Adım



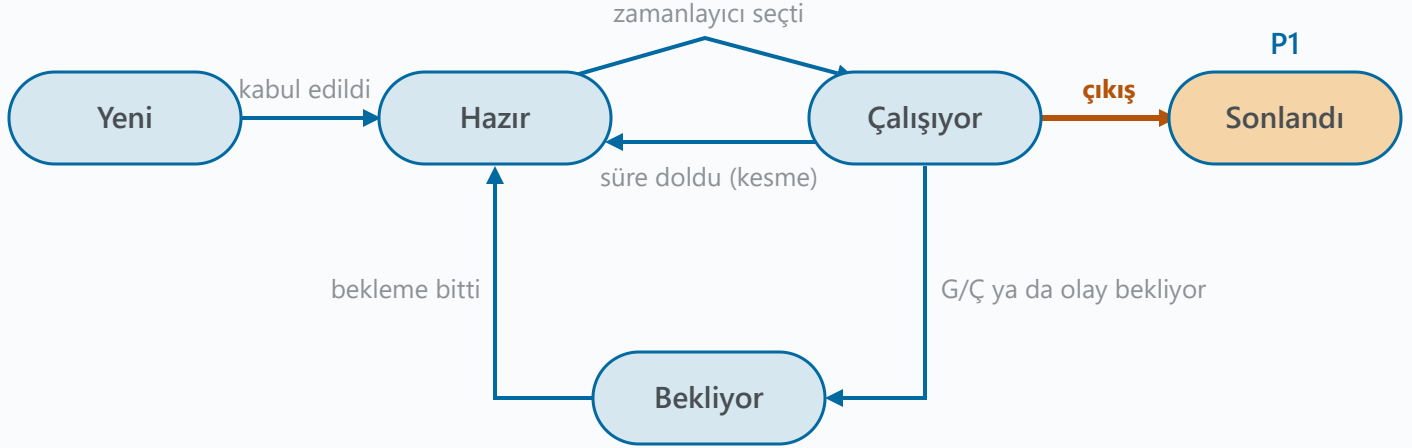
Çalışıyor → **Hazır**: P1'in **zaman dilimi doldu**; zamanlayıcı kesmesiyle CPU başka sürece verilir.

Süreç Durum Geçişleri Adım Adım



Hazır → **Çalışıyor**: P1 tekrar seçilir ve işinin kalanını yürütür.

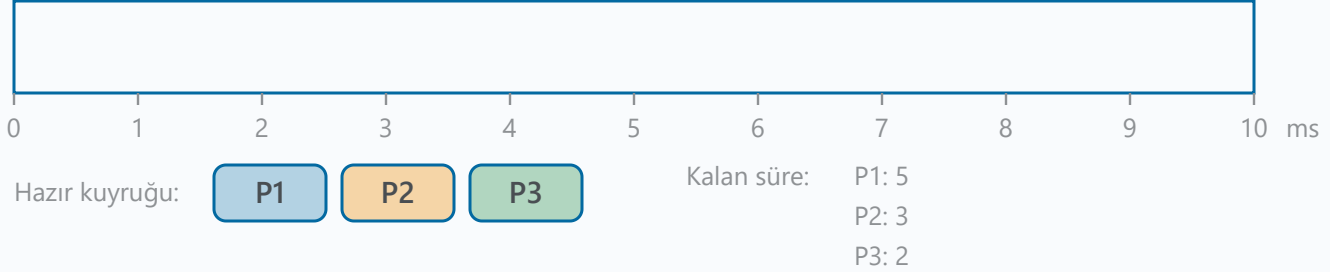
Süreç Durum Geçişleri Adım Adım



Çalışıyor → **Sonlandı**: P1 işini bitirip **çıkartır**; işletim sistemi belleğini ve kaynaklarını geri alır.

Zaman Paylaşımı: Döner Sıralı CPU Dağıtımı

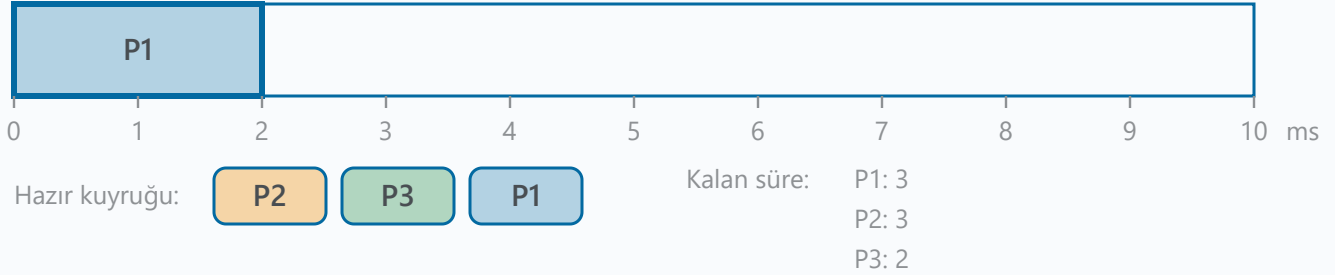
CPU zaman çizelgesi (Gantt)



Üç süreç hazır: P1 5 ms, P2 3 ms, P3 2 ms iş istiyor. **Döner sıralı** (round robin) zamanlama, her birine sırayla en çok **2 ms** verir.

Zaman Paylaşımı: Döner Sıralı CPU Dağıtımı

CPU zaman çizelgesi (Gantt)



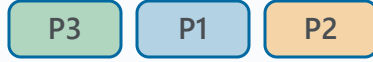
$t = 0-2$: CPU **P1**'de. Zaman dilimi doldu; P1 kuyruğun sonuna gider (kalan 3 ms). Bağlam değiştirilir.

Zaman Paylaşımı: Döner Sıralı CPU Dağıtımı

CPU zaman çizelgesi (Gantt)



Hazır kuyruğu:

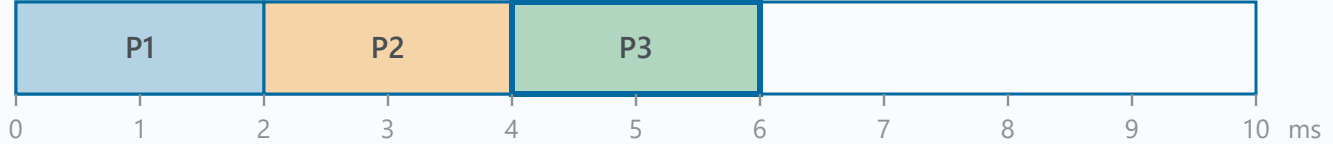


Kalan süre: P1: 3
P2: 1
P3: 2

$t = 2-4$: CPU **P2**'de. Zaman dilimi doldu; P2 kuyruğun sonuna gider (kalan 1 ms). Bağlam değiştirilir.

Zaman Paylaşımı: Döner Sıralı CPU Dağıtımı

CPU zaman çizelgesi (Gantt)



Hazır kuyruğu:

P1

P2

Kalan süre: P1: 3

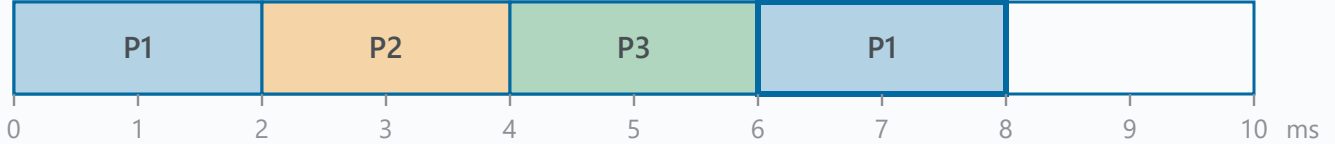
P2: 1

P3: 0 (bitti t=6)

t = 4–6: CPU **P3**'de. P3 işini bitirdi (t = 6) ve sistemden çıkar.

Zaman Paylaşımı: Döner Sıralı CPU Dağıtımı

CPU zaman çizelgesi (Gantt)



Hazır kuyruğu:



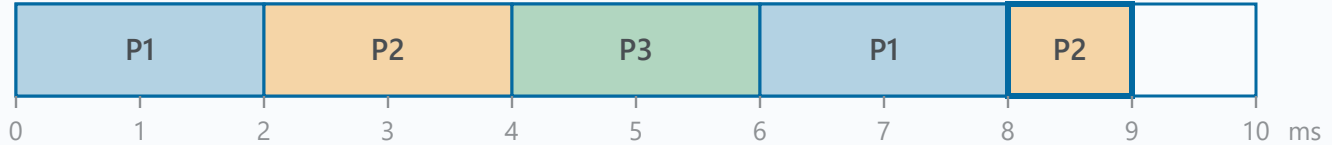
Kalan süre: P1: 1
P2: 1

P3: 0 (bitti t=6)

t = 6–8: CPU **P1**'de. Zaman dilimi doldu; P1 kuyruğun sonuna gider (kalan 1 ms). Bağlam değiştirilir.

Zaman Paylaşımı: Döner Sıralı CPU Dağıtımı

CPU zaman çizelgesi (Gantt)



Hazır kuyruğu:

P1

Kalan süre: P1: 1

P2: 0 (bitti t=9)

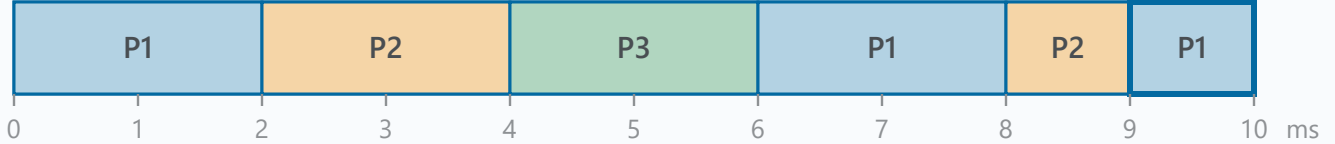
P3: 0 (bitti t=6)

t = 8–9: CPU **P2**'de. P2 işini bitirdi (t = 9) ve sistemden çıkar.

Adım 6 / 7

Zaman Paylaşımı: Döner Sıralı CPU Dağıtımı

CPU zaman çizelgesi (Gantt)



Hazır kuyruğu: boş

Kalan süre: **P1: 0 (bitti t=10)**
P2: 0 (bitti t=9)
P3: 0 (bitti t=6)

t = 9–10: CPU **P1**'de. P1 işini bitirdi (t = 10) ve sistemden çıkar. Bitiş: P3 = 6, P2 = 9, P1 = 10 ms. Hızlı geçişler sayesinde üçü de "aynı anda" çalışıyor gibi görünür.

Kilitlenme (Deadlock)



düz ok: kaynak sürecin elinde · kesikli ok: süreç kaynağı bekliyor

Süreç A **yazıcıyı** aldı ve kullanıyor (düz ok).

Adım 1 / 5

Karşılıklı dışlama

tek sahip

Tut ve bekle

bırakmadan bekler

Ön alınamazlık

zorla alınamaz

Dairesel bekleme

halka biçiminde

Dört koşulun **hepsi** gerekir. Çözüm fikri: kaynakları herkes aynı **sırayla** istesin (önce Yazıcı, sonra Dosya); dairesel bekleme oluşmaz.

Kilitlenme (Deadlock)



düz ok: kaynak sürecin elinde · kesikli ok: süreç kaynağı bekliyor

Süreç B **dosyayı** kilitledi ve kullanıyor.

Adım 2 / 5

Karşılıklı dışlama

tek sahip

Tut ve bekle

bırakmadan bekler

Ön alınamazlık

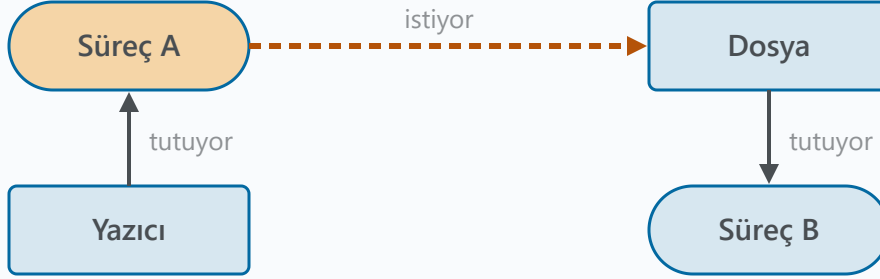
zorla alınamaz

Dairesel bekleme

halka biçiminde

Dört koşulun **hepsi** gerekir. Çözüm fikri: kaynakları herkes aynı **sırayla** istesin (önce Yazıcı, sonra Dosya); dairesel bekleme oluşmaz.

Kilitlenme (Deadlock)



düz ok: kaynak sürecin elinde · kesikli ok: süreç kaynağı bekliyor

Süreç A, yazıcıyı bırakmadan **dosyayı** istiyor (kesikli ok); dosya B'de olduğu için bekler.

Adım 3 / 5

Karşılıklı dışlama

tek sahip

Tut ve bekle

bırakmadan bekler

Ön alınamazlık

zorla alınamaz

Dairesel bekleme

halka biçiminde

Dört koşulun **hepsi** gerekir. Çözüm fikri: kaynakları herkes aynı **sırayla** istesin (önce Yazıcı, sonra Dosya); dairesel bekleme oluşmaz.

Kilitlenme (Deadlock)



düz ok: kaynak sürecin elinde · kesikli ok: süreç kaynağı bekliyor

Süreç B de dosyayı bırakmadan **yazıcıyı** istiyor; yazıcı A'da olduğu için bekler.

Adım 4 / 5

Karşılıklı dışlama

tek sahip

Tut ve bekle

bırakmadan bekler

Ön alınamazlık

zorla alınamaz

Dairesel bekleme

halka biçiminde

Dört koşulun **hepsi** gerekir. Çözüm fikri: kaynakları herkes aynı **sırayla** istesin (önce Yazıcı, sonra Dosya); dairesel bekleme oluşmaz.

Kilitlenme (Deadlock)



düz ok: kaynak sürecin elinde · kesikli ok: süreç kaynağı bekliyor

A → Dosya → B → Yazıcı → A: **dairesel bekleme**. İkisi de bırakmadığı için ikisi de sonsuza dek bekler: **kilitlenme**.

Adım 5 / 5

Karşılıklı dışlama

tek sahip

Tut ve bekle

bırakmadan bekler

Ön alınamazlık

zorla alınamaz

Dairesel bekleme

halka biçiminde

Dört koşulun **hepsi** gerekir. Çözüm fikri: kaynakları herkes aynı **sırayla** istesin (önce Yazıcı, sonra Dosya); dairesel bekleme oluşmaz.

Bellek Yönetimi

Ayırma

Her programa gereksinimine göre bellek alanı sağlama.

Koruma

Bir sürecin başka sürecin belleğini izinsiz değiştirmesini önleme.

Sanal bellek

Programlara fiziksel RAM'den bağımsız adres alanı sunma.

Sayfalama

Belleği sabit boyutlu parçalarla yönetme.

Takas

Gerektiğinde bazı verileri geçici olarak diske taşıma.

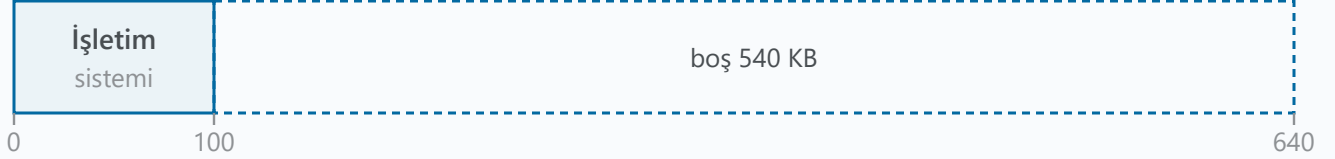
Paylaşım

Ortak kod veya veriyi kontrollü biçimde kullanma.

Bellek Yönetimi Adım Adım: Ayırma, Koruma, Takas

RAM (640 KB)

Disk (takas alanı)



Taban / limit yazmaçları: henüz kullanıcı süreci yok

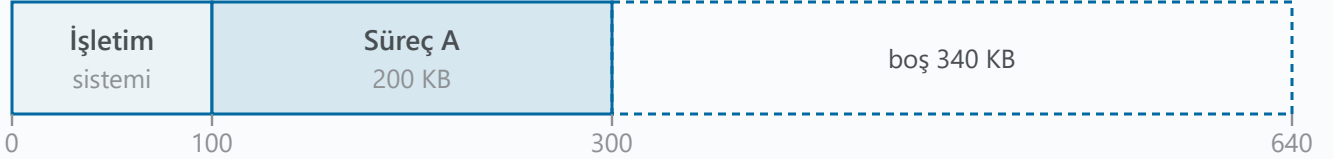
RAM'in başı çekirdeğe ayrılmıştır; kalan alan boştur.

Başlangıç: RAM'in başında **işletim sistemi (çekirdek)** vardır; geri kalanı boştur.

Bellek Yönetimi Adım Adım: Ayırma, Koruma, Takas

RAM (640 KB)

Disk (takas alanı)



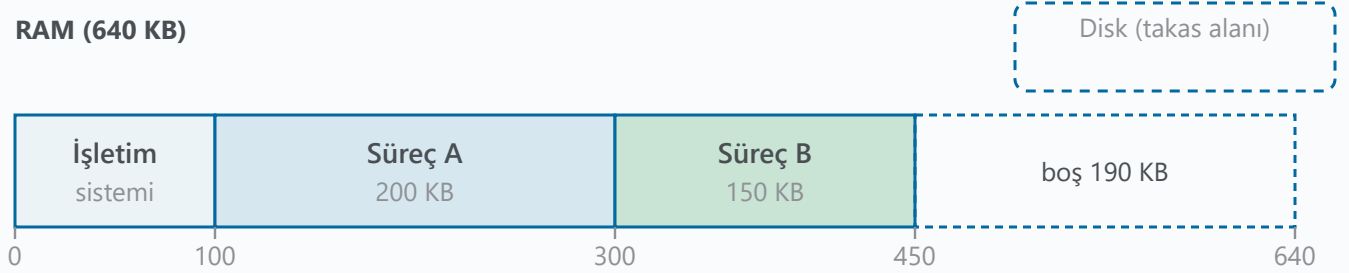
Süreç A → taban = 100, limit = 200

Taban: bölgenin başlangıcı. Limit: bölgenin uzunluğu (KB).

Ayırma: A için 100–300 arası 200 KB ayrılır. Basit modelde bölge **taban** ve **limit** yazmaçlarıyla tanımlanır (gerçekte sayfalama kullanılır).

Bellek Yönetimi Adım Adım: Ayırma, Koruma, Takas

RAM (640 KB)

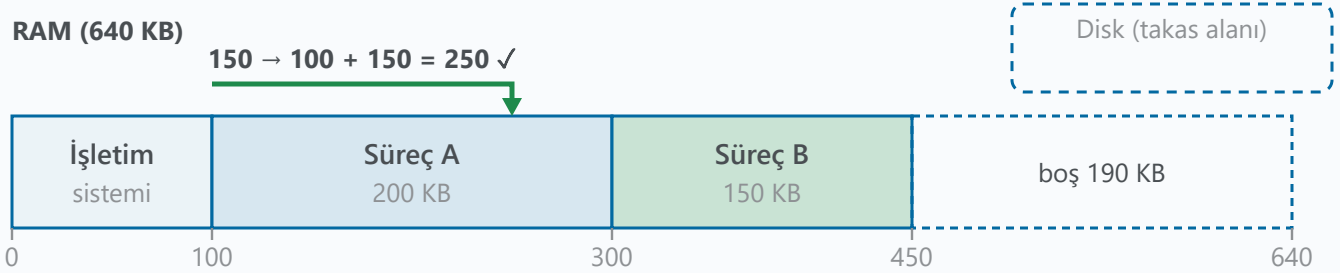


A: taban = 100, limit = 200 B: taban = 300, limit = 150

Her sürecin kendi bölgesi vardır; bölgeler çakışmaz.

B için 300–450 arası 150 KB ayrılır. Her sürecin yazmaç çifti ayrıdır; süreç değişince işletim sistemi bunları yükler.

Bellek Yönetimi Adım Adım: Ayırma, Koruma, Takas

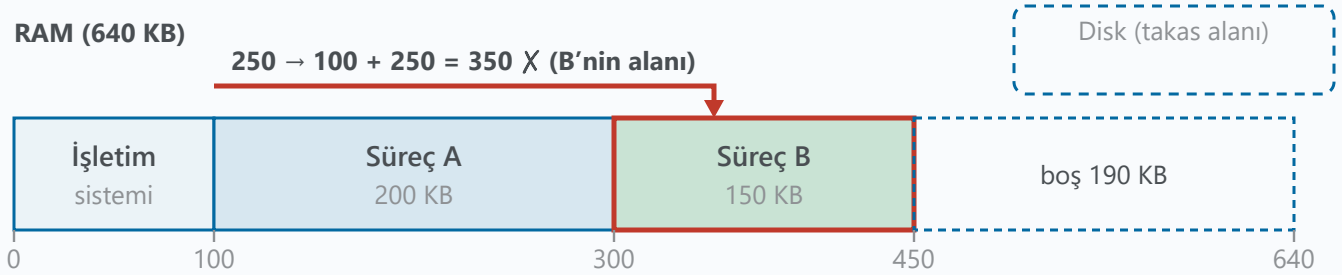


A çalışıyor, mantıksal adres 150 isteniyor

$150 < \text{limit (200)} \rightarrow \text{fiziksel adres} = \text{taban} + 150 = 250 \checkmark$

Geçerli erişim: A'nın adresi 150, limitin (200) altındadır; donanım **taban + 150 = 250** fiziksel adresine gider.

Bellek Yönetimi Adım Adım: Ayırma, Koruma, Takas

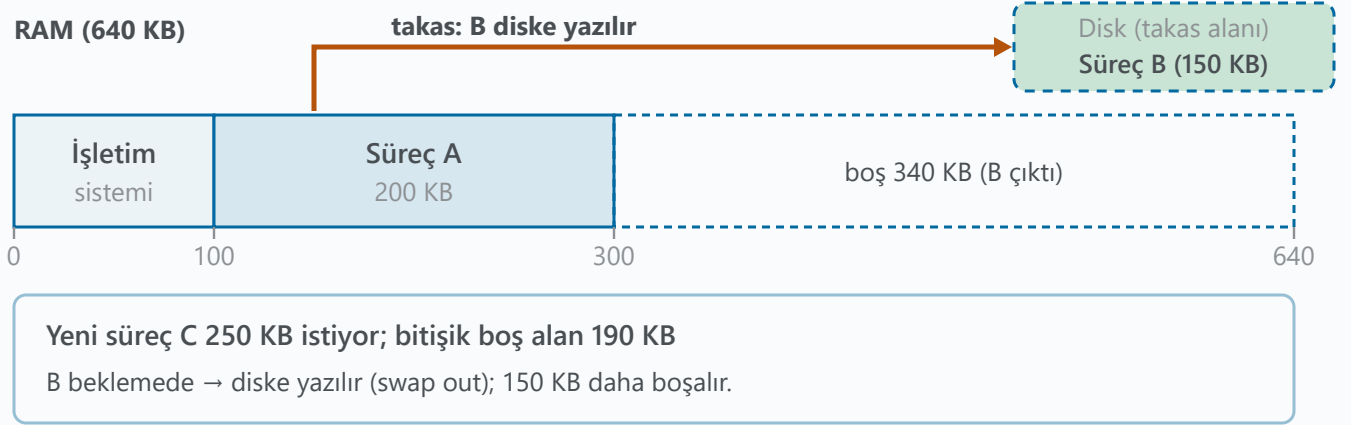


A çalışıyor, mantıksal adres 250 isteniyor

$250 \geq \text{limit (200)} \rightarrow \text{ihlal } X: \text{ donanım kesmesi, erişim engellenir}$

Koruma ihlali: A 250 ister, $250 \geq 200$ olduğu için donanım erişimi keser; işletim sistemi A'yı sonlandırır ("erişim ihlali / segmentation fault"). B'nin belleği korunur.

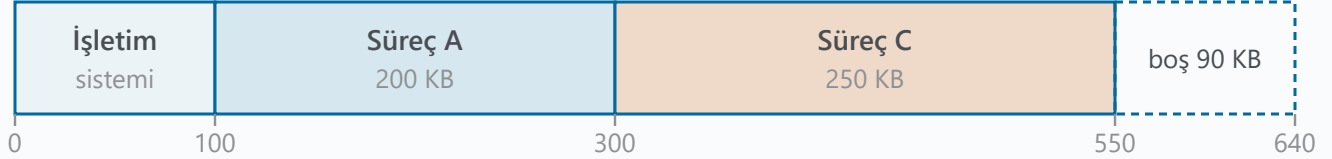
Bellek Yönetimi Adım Adım: Ayırma, Koruma, Takas



Takas: C'ye yer yoktur ($190 \text{ KB} < 250 \text{ KB}$). Beklemedeki B diske yazılır; RAM'de 340 KB bitişik boş alan oluşur. Disk RAM'den çok yavaştır.

Bellek Yönetimi Adım Adım: Ayırma, Koruma, Takas

RAM (640 KB)

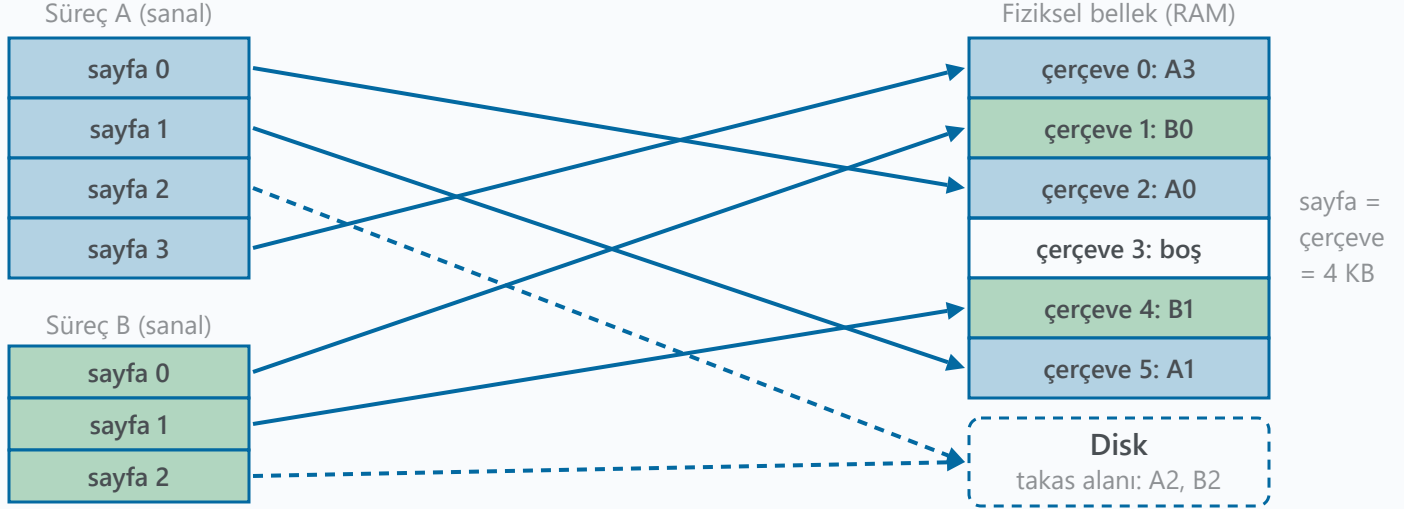


Süreç C → taban = 300, limit = 250

B diskte bekler; çalışacağı zaman geri yüklenir (swap in).

C, 300–550 arasına yüklenir. B diskte bekler ve yeniden çalışacağında RAM'e geri alınır; yer yoksa başka bir süreç diske yazılır.

Sanal Bellek ve Sayfalama



- Her süreç 0'dan başlayan kendi **sanal adres** alanını görür; bellek 4 KB'lık **sayfalara** bölünür ve sayfa tablosu her sayfanın hangi **çerçevede** olduğunu tutar.
- Sayfalar RAM'de ardışık olmak zorunda değildir; az kullanılan sayfa **diske** (takas) çıkarılabilir.

Adres Çevirisi Adım Adım: 4100 ve 8200

Sanal adres

4100

Sayfa tablosu (Süreç A)

0	çerçeve 2
1	çerçeve 5
2	diskte (yok)
3	çerçeve 0

RAM

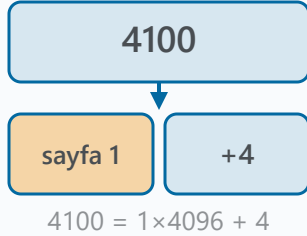
0: A3
1: B0
2: A0
3: boş
4: B1
5: A1

Disk (takas)

Süreç A, **4100** sanal adresindeki veriyi okumak istiyor. İşlemci'deki bellek yönetim birimi (MMU) bu adresi çevirmeli.

Adres Çevirisi Adım Adım: 4100 ve 8200

Sanal adres



Sayfa tablosu (Süreç A)

0	çerçeve 2
1	çerçeve 5
2	diskte (yok)
3	çerçeve 0

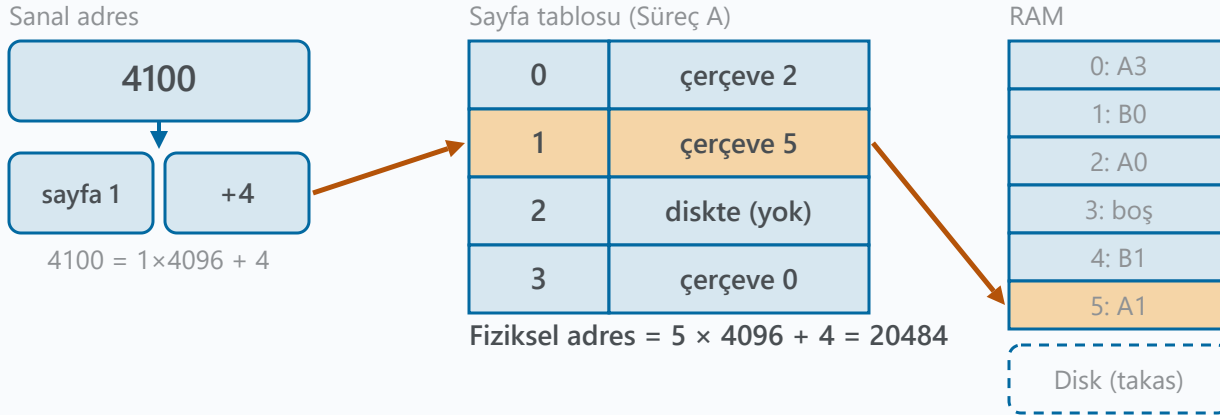
RAM

0: A3
1: B0
2: A0
3: boş
4: B1
5: A1

Disk (takas)

Adres ikiye ayrılır: **sayfa** = $4100 / 4096 = 1$, **uzaklık** = $4100 \bmod 4096 = 4$.

Adres Çevirisi Adım Adım: 4100 ve 8200



Tablo: sayfa 1 → çerçeve 5. **Fiziksel adres = $5 \times 4096 + 4 = 20484$.**

Adres Çevirisi Adım Adım: 4100 ve 8200

Sanal adres

8200

Sayfa tablosu (Süreç A)

0	çerçeve 2
1	çerçeve 5
2	diskte (yok)
3	çerçeve 0

RAM

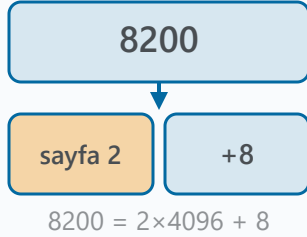
0: A3
1: B0
2: A0
3: boş
4: B1
5: A1

Disk (takas)

Süreç A, **8200** sanal adresindeki veriyi okumak istiyor. İşlemciadaki bellek yönetim birimi (MMU) bu adresi çevirmeli.

Adres Çevirisi Adım Adım: 4100 ve 8200

Sanal adres



Sayfa tablosu (Süreç A)

0	çerçeve 2
1	çerçeve 5
2	diskte (yok)
3	çerçeve 0

RAM

0: A3
1: B0
2: A0
3: boş
4: B1
5: A1

Disk (takas)

Adres ikiye ayrılır: **sayfa** = $8200 / 4096 = 2$, **uzaklık** = $8200 \bmod 4096 = 8$.

Adres Çevirisi Adım Adım: 4100 ve 8200

Sanal adres

8200

sayfa 2 +8

$$8200 = 2 \times 4096 + 8$$

Sayfa tablosu (Süreç A)

0	çerçeve 2
1	çerçeve 5
2	diskte (yok)
3	çerçeve 0

SAYFA HATASI (page fault)

RAM

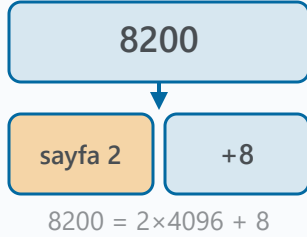
0: A3
1: B0
2: A0
3: boş
4: B1
5: A1

Disk (takas)

Tabloda sayfa 2 **RAM'de değil** → sayfa hatası. Program durdurulur, işletim sistemi devreye girer.

Adres Çevirisi Adım Adım: 4100 ve 8200

Sanal adres



Sayfa tablosu (Süreç A)

0	çerçeve 2
1	çerçeve 5
2	çerçeve 3
3	çerçeve 0

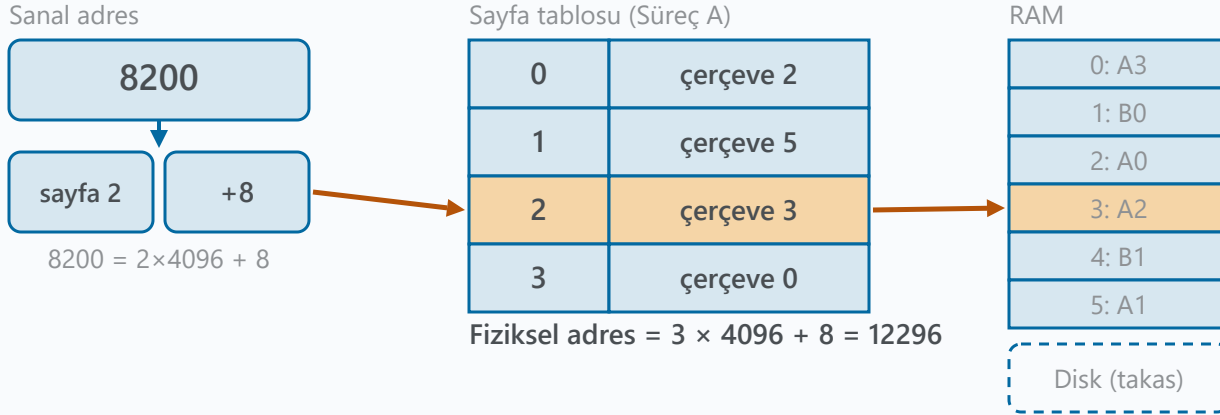
RAM

0: A3
1: B0
2: A0
3: A2
4: B1
5: A1

Disk (takas)

İşletim sistemi sayfayı diskten boş **çerçeve 3**'e yükler ve sayfa tablosunu günceller.

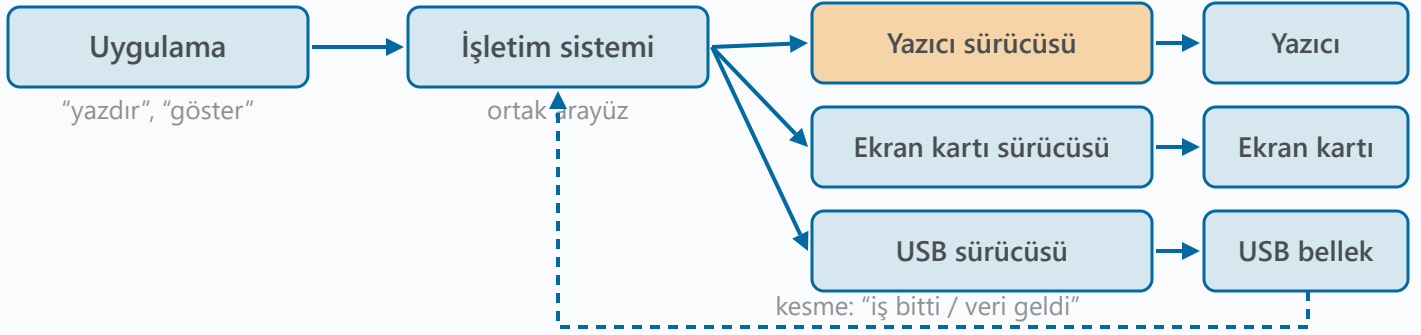
Adres Çevirisi Adım Adım: 4100 ve 8200



Tablo: sayfa 2 → çerçeve 3. **Fiziksel adres = $3 \times 4096 + 8 = 12296$** . Komut yeniden çalışır ve bu kez başarılı olur.

Aygıtlar ve Sürücüler

Donanım aygıtlarının komutları ve özellikleri farklıdır. Aygıt sürücüsü, işletim sisteminin ortak isteklerini donanıma özgü işlemlere çevirir.



Tak ve çalıştır

Uyumlu aygıtı algılayıp uygun sürücüyü yapılandırma.

Kesme

Aygıtın işlemciye olay bildirmesi.

Tampon

Hızları farklı bileşenler arasında geçici veri tutma.

Dosya Sistemi

Dosya sistemi, verinin depolama aygıtında nasıl adlandırıldığını, düzenlendiğini ve bulunduğunu tanımlar.

Dosya

Adı ve içeriği olan veri birimi.

Klasör

Dosya ve alt klasörleri gruplar.

Yol

Dosyanın klasör yapısındaki konumunu belirtir.

Meta veri

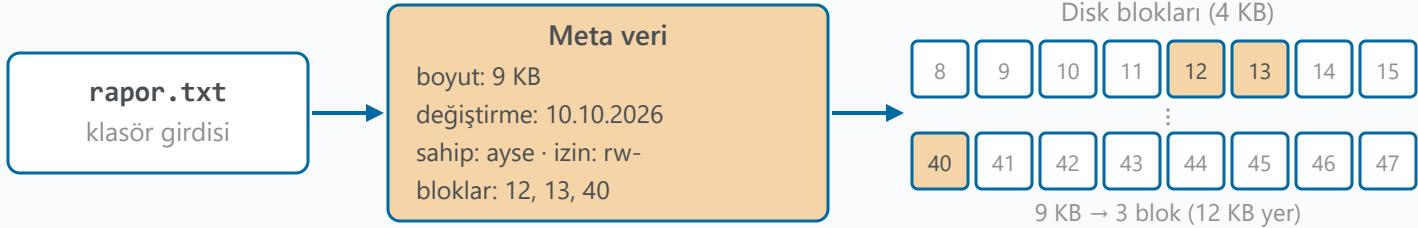
Boyut, tarih, sahip ve izin gibi özellikler.

İzinler

Okuma, yazma ve çalıştırma yetkileri.

Günlükleme (journaling)

Beklenmeyen kesintilerden sonra tutarlılığı destekler.

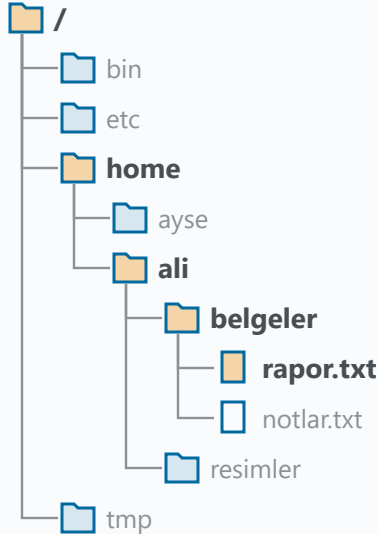


Dosya Sistemi Türleri

Dosya sistemi	Platform	Günlükleme	Ne zaman kullanılır?
NTFS	Windows (macOS yalnızca okur)	Var	Windows sistem diski; büyük dosya, izin ve şifreleme gerektiğinde
FAT32	Hemen her aygıt ve işletim sistemi	Yok	Eski aygıtlar, kamera, basit bellek kartı; tek dosya en çok 4 GB
exFAT	Windows, macOS ve güncel Linux	Yok	Büyük dosyalı USB bellek ve SD kart; platformlar arası taşıma
ext4	Linux	Var	Linux sistem ve veri diski
APFS	macOS, iOS, iPadOS	Klasik günlük yok; kopya-yaz ile tutarlılık	Apple cihazlarının varsayılan dosya sistemi

Örnek: 5 GB'lık bir video dosyası FAT32 biçimli USB belleğe kopyalanamaz; bellek exFAT veya NTFS'ye biçimlendirilmelidir (biçimlendirme içeriği siler).

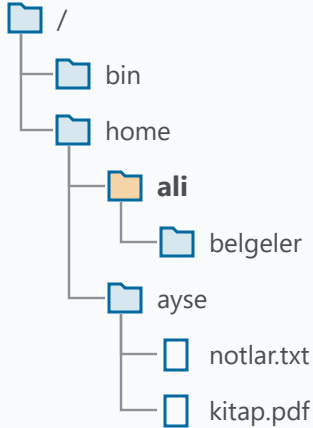
Dosya Sistemi Ağacı ve Yol



/home/ali/belgeler/rapor.txt

- Dosya sistemi ağaç biçimindedir: **kök** en üstte, klasörler dallar, dosyalar yapraklardır.
- **Mutlak yol** kökten başlar; **görelî yol** geçerli klasöre göredir (ali'deyken: `belgeler/rapor.txt`).
- Windows'ta aynı fikir sürücü harfiyle yazılır: `C:\Users\Ali\Documents\rapor.txt` .

Görelî Yol Adım Adım: ../ayse/notlar.txt



Görelî yol (cwd = `/home/ali`)

`..` / `ayse` / `notlar.txt`

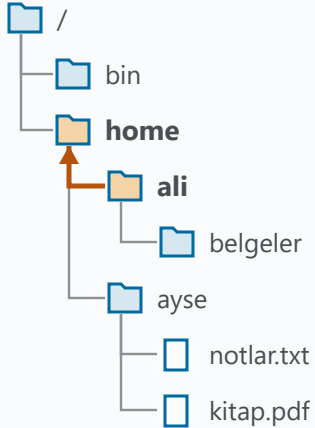
Başlangıç konumu

`/home/ali`

Komut `/home/ali` klasöründe çalışıyor (geçerli klasör). Görelî yol bu klasörden başlar:

`../ayse/notlar.txt` üç parçadır.

Görelî Yol Adım Adım: ../ayse/notlar.txt



Görelî yol (cwd = `/home/ali`)

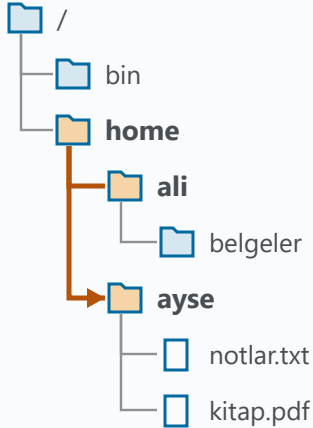
`..` / `ayse` / `notlar.txt`

".." → bir üst klasör

`/home`

`..` bir üst klasör demektir: `/home/ali` → `/home`.

Görelî Yol Adım Adım: ../ayse/notlar.txt



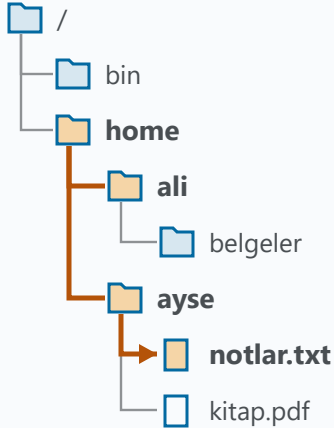
Görelî yol (cwd = /home/ali)



"ayse" → alt klasöre in
/home/ayse

ayse, /home içindeki alt klasördür: /home → **/home/ayse**.

Görelî Yol Adım Adım: ../ayse/notlar.txt



Görelî yol (cwd = /home/ali)

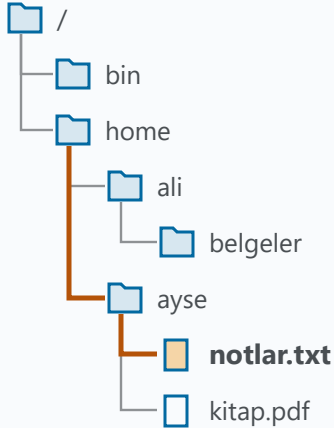


"notlar.txt" → dosya bulundu

/home/ayse/notlar.txt

Son parça dosya adıdır: /home/ayse içinde **notlar.txt** bulunur.

Görelî Yol Adım Adım: ../ayse/notlar.txt



Görelî yol (cwd = /home/ali)



Çözülen mutlak yol

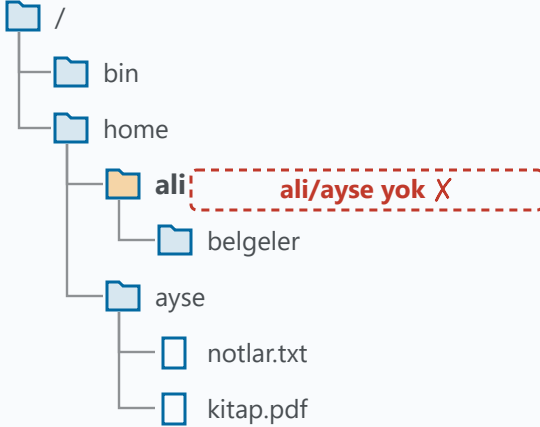
/home/ayse/notlar.txt

Windows'ta aynı yol: ..\ayse\notlar.txt

"." geçerli klasör, ".." üst klasördür.

Sonuç: **/home/ayse/notlar.txt**. Görelî yol, komutun çalıştığı klasöre bağlıdır; mutlak yol her yerden aynı dosyayı gösterir.

Görelî Yol Adım Adım: ../ayse/notlar.txt



Görelî yol (cwd = /home/ali)

ayse

/ notlar.txt

Çözülen yol (".." yok)

/home/ali/ayse/notlar.txt

Bu klasör yok: "Böyle bir dosya ya da dizin yok" hatası alınır.

Sık hata: baştaki "../" unutulursa yol **/home/ali/ayse/notlar.txt** olarak çözülür ve dosya bulunamaz.

Kullanıcı Arayüzleri

Komut İstemi

```
C:\> dir
rapor.docx notlar.txt
C:\> copy rapor.docx E:\
1 dosya kopyalandı.
C:\> _
```

Dosya Gezgini



rapor



notlar



resimler



GUI (grafik kullanıcı arayüzü)

Pencere, simge, menü ve fareyle etkileşim.

CLI

Metin komutlarıyla hızlı, tekrarlanabilir, otomasyona uygun.

Dokunmatik arayüz

Dokunma, kaydırma ve hareketlerle etkileşim.

Erişilebilirlik

Ekran okuyucu, büyütme, yüksek kontrast.

Komut Satırı Temelleri

İş	Linux / macOS	Windows (cmd)
Geçerli klasörü göster	<code>pwd</code>	<code>cd</code> (parametresiz)
Klasör içeriğini listele	<code>ls</code>	<code>dir</code>
Klasör değiştir	<code>cd klasör</code>	<code>cd klasör</code>
Klasör oluştur	<code>mkdir yeni</code>	<code>mkdir yeni</code>
Dosya kopyala	<code>cp a.txt b.txt</code>	<code>copy a.txt b.txt</code>
Dosya sil	<code>rm a.txt</code>	<code>del a.txt</code>

chmod izin biçimi

Üç grup için **r**=4 (oku), **w**=2 (yaz), **x**=1 (çalıştır) toplanır: sahip · grup · diğerleri.

`chmod 644 dosya`

6 = rw- (sahip) · 4 = r-- (grup) · 4 = r-- (diğer)

	sahip			grup			diğer		
izin	r	w	-	r	-	-	r	-	-
bit	1	1	0	1	0	0	1	0	0
değer	4	2	1	4	2	1	4	2	1
toplam	4+2 = 6			4			4		

`rm` ve `del` dosyayı Geri Dönüşüm Kutusu'na taşımaz; komutu çalıştırmadan önce yolu ve adı kontrol edin.

Kullanıcı ve Yetki Yönetimi



- **Standart hesap** günlük kullanımda sistem değişikliklerini sınırlar; **yönetici hesabı** sistem genelinde değişiklik yapabilir.
- **En az ayrıcalık:** kullanıcıya yalnızca işi için gerekli yetki verilir.
- **Kayıt tutma**, önemli olayların sonradan incelenebilmesini sağlar.
- Örnekte standart kullanıcı kendi dosyasını açabilir, sistem ayarını değiştiremez.

Kişisel Bilgisayar Sistemleri

Windows

Geniş donanım ve uygulama ekosistemi.

macOS

Apple donanımıyla bütünleşik masaüstü sistemi.

Linux dağıtımları

Farklı masaüstü ve kullanım amaçlarına göre özelleştirilebilir sistemler.

ChromeOS

Web ve bulut tabanlı kullanıma odaklanan sistem.

Mobil İşletim Sistemleri

Android

Farklı üreticilerin cihazlarında çalışan geniş mobil ekosistem.

iOS / iPadOS

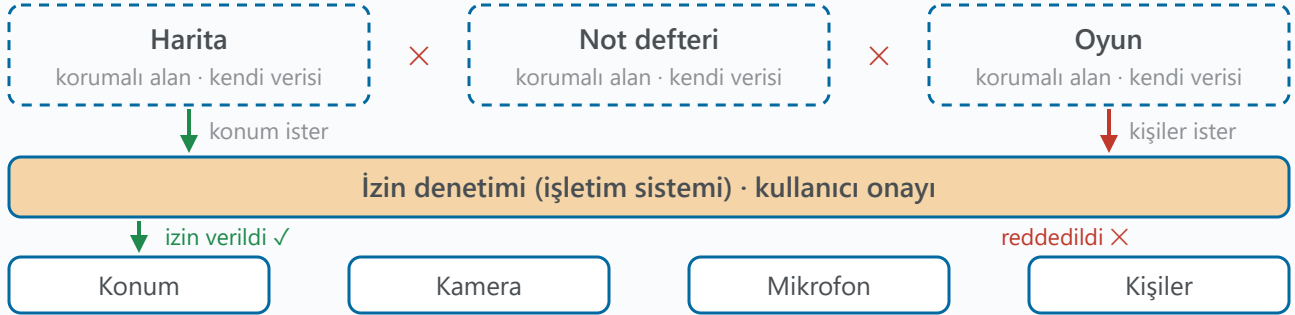
Apple mobil cihazları için bütünlük sistem.

Mobil öncelikler

Dokunmatik arayüz, enerji yönetimi, sensörler ve kablosuz bağlantı.

Uygulama yalıtımı

Uygulamaların erişimini izinler ve korumalı alanlarla sınırlar.



Sunucu İşletim Sistemleri

Çok kullanıcılı çalışma

Çok sayıda kullanıcı ve hizmeti aynı anda yönetir.

Ağ hizmetleri

Web, veri tabanı, dosya ve kimlik hizmetleri sunar.

Süreklilik

Uzun süre kesintisiz çalışma ve uzaktan yönetim hedeflenir.

Güvenlik

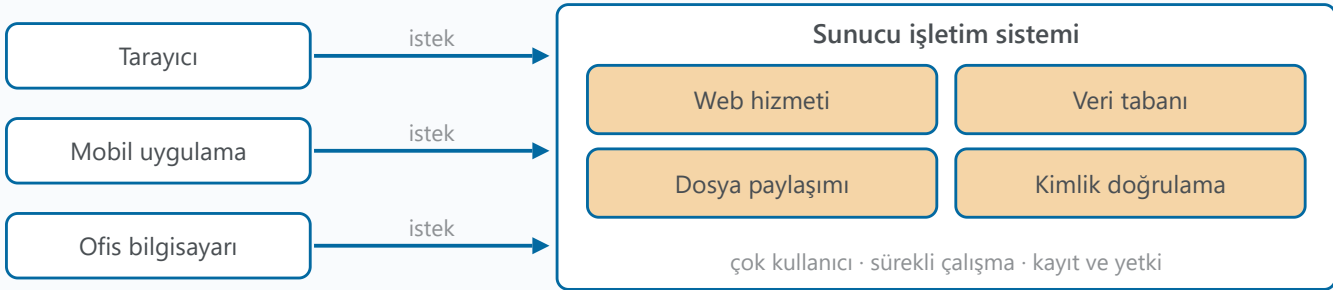
Yetki, kayıt, güncelleme ve ağ denetimi ön plandadır.

Örnekler

Linux dağıtımları, Windows Server ve Unix türevleri.

Otomasyon

Komut satırı ve yapılandırma araçlarıyla tekrarlanabilir yönetim.



Gömülü ve Gerçek Zamanlı Sistemler

Gömülü işletim sistemi

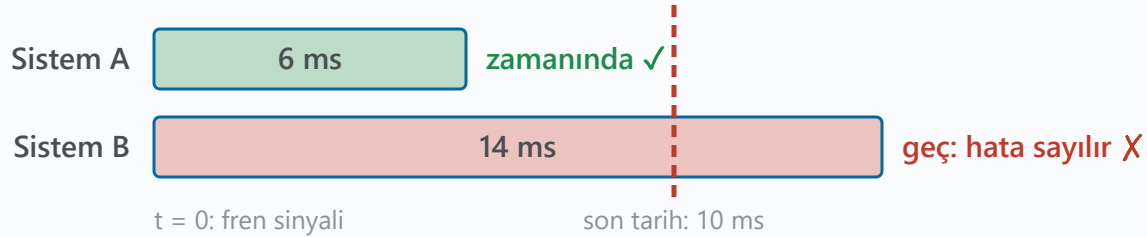
Belirli bir cihaz görevi için sınırlı işlemci, bellek ve enerjiyle çalışır.

Örnek: yönlendirici, otomobil kontrol birimi, tıbbi cihaz.

Gerçek zamanlı işletim sistemi

Bir işlemin yalnızca doğru sonucu değil, belirlenen süre içinde sonucu üretmesini hedefler.

Örnek: endüstriyel kontrol ve güvenlik sistemleri.



İşletim Sistemi Seçimi

Gereksinim	Değerlendirilecek ölçüt
Uygulama	Gerekli yazılımların ve sürümlerin bulunması
Donanım	Sürücü ve mimari desteği
Güvenlik	Güncelleme süresi, yetki modeli ve yönetim araçları
Kullanılabilirlik	Arayüz, erişilebilirlik ve öğrenme gereksinimi
Maliyet	Lisans, destek, eğitim ve bakım toplamı

Açık Kaynak Yazılım

Kaynak kodu

Lisans koşulları çerçevesinde incelenebilir.

Değiştirme

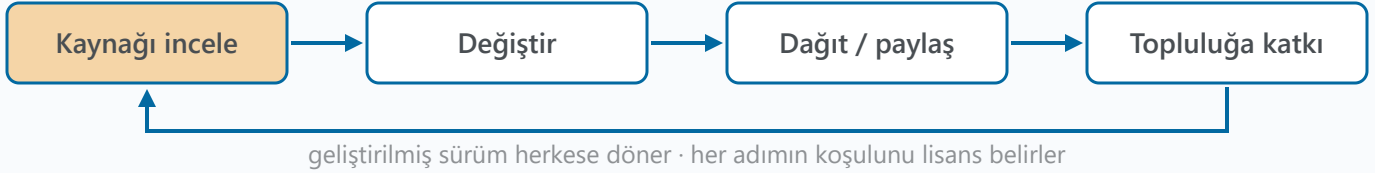
Lisans izin veriyorsa ihtiyaçlara göre uyarlanabilir.

Dağıtım

Değiştirilmiş veya özgün sürümün paylaşımı lisansa bağlıdır.

Topluluk

Geliştirme ve destek bireyler, kurumlar ve şirketler tarafından yürütülebilir.



Açık kaynak, otomatik olarak ücretsiz veya desteksiz demek değildir. Hakları ve yükümlülükleri lisans belirler.

Ticari ve Kapalı Kaynak Yazılım

Kullanım hakkı

Yazılım satın alınmaz; çoğunlukla belirli koşullarla kullanım lisansı alınır.

Kaynak kodu

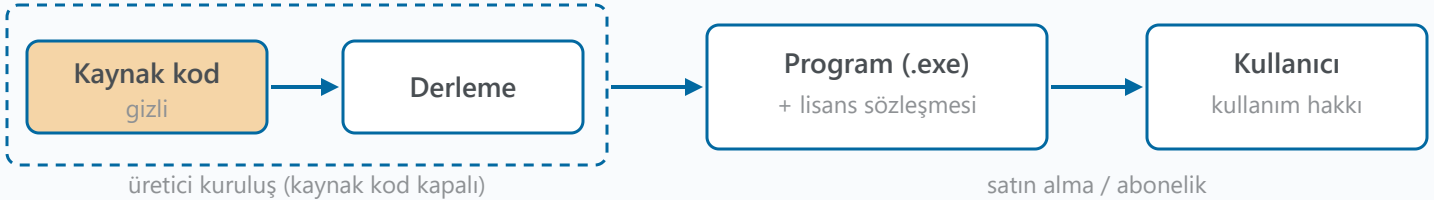
Genellikle geliştirici kuruluş tarafından kapalı tutulur.

Destek

Resmî destek ve hizmet düzeyi sözleşmeleri sunulabilir.

Maliyet

Lisans, abonelik, bakım ve yükseltme ücretleri olabilir.



Lisans Modellerini Karşılaştırma

Ölçüt	Açık kaynak	Ticari / kapalı kaynak
Kaynak erişimi	Lisansla açık	Genellikle kapalı
Özelleştirme	Lisans kapsamında mümkün	Üreticinin sunduğu seçeneklerle sınırlı
Destek	Topluluk veya ücretli sağlayıcı	Üretici veya yetkili ortak
Maliyet	Lisans ücretsiz olabilir; işletme maliyeti devam eder	Lisans veya abonelik gerekebilir

Güncelleme ve Destek

Güvenlik güncellemesi

Bilinen açıkları azaltır.

Özellik güncellemesi

Yeni işlev ve arayüz değişiklikleri getirir.

Sürücü güncellemesi

Donanım uyumluluğu veya kararlılığını geliştirebilir.

Destek süresi

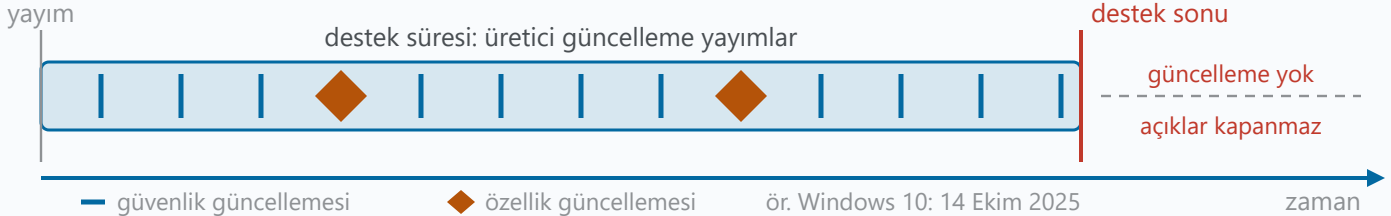
Üreticinin güncelleme sağladığı zaman aralığıdır.

Yedekleme

Büyük değişikliklerden önce geri dönüş imkanı sağlar.

Test

Kurumsal ortamlarda güncellemeler yaygın dağıtımdan önce denir.



Sanallaştırma ve Kapsayıcılar

Sanal makine

Bir fiziksel bilgisayarda yalıtılmış işletim sistemi örnekleri çalıştırır.

Hipervizör

Sanal makinelere işlemci, bellek ve aygıt kaynakları sağlar.

Kapsayıcı

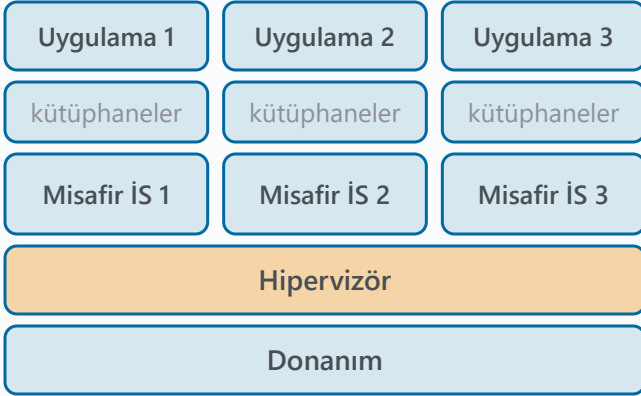
Uygulamaları ve bağımlılıklarını süreç düzeyinde yalıtır; çekirdeği paylaşır.

Kullanım

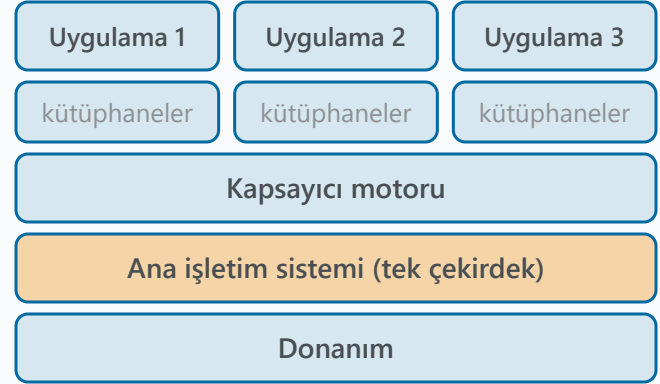
Test, sunucu birleştirme, bulut ve taşınabilir uygulama dağıtımı.

Sanal Makine ile Kapsayıcı Karşılaştırması

Sanal makineler



Kapsayıcılar



Sanal makine

Tam bir misafir işletim sistemi taşır: güçlü yalıtım, ama daha çok bellek ve daha yavaş açılış.

Kapsayıcı

Yalnızca uygulama ve bağımlılıkları; çekirdek ortak: hafif ve hızlı, yalıtım daha zayıf.

Bölüm Özeti

- Yazılım, donanımın gerçekleştireceği işlemleri tanımlar.
- İşletim sistemi uygulamalarla donanım arasında yönetim ve soyutlama katmanı oluşturur.
- Çekirdek süreç, bellek, aygıt ve dosya kaynaklarını yönetir.
- Kişisel, mobil, sunucu, gömülü ve gerçek zamanlı sistemlerin öncelikleri farklıdır.
- İşletim sistemi seçiminde uygulama, donanım, güvenlik, destek ve toplam maliyet birlikte değerlendirilir.
- Açık ve kapalı kaynak haklarını kullanılan lisans belirler.

Kontrol Soruları

1. Sistem yazılımı ile uygulama yazılımı arasındaki fark nedir?
2. Çekirdek neden ayrıcalıklı modda çalışır?
3. Program ve süreç kavramları nasıl ayrılır?
4. Sanal bellek hangi problemi çözer?
5. Sunucu ve mobil işletim sistemlerinin öncelikleri nasıl değişir?
6. Açık kaynak yazılım neden her zaman ücretsiz olmak zorunda değildir?

Alıştırmalar

1. Döner sıralı zamanlama, zaman dilimi 2 ms: **P1 = 4, P2 = 2, P3 = 3 ms**. Gantt çizelgesini çizin ve her sürecin bitiş zamanını bulun.
2. Sayfa boyutu 4096 bayt. Sanal adres **12300** hangi sayfada ve sayfa içinde hangi uzaklıktadır? Sayfa 3, çerçeve 1'deyse fiziksel adres nedir?
3. Klavyeden girdi bekleyen bir süreç hangi durumdadır? Tuşa basılınca hangi duruma, sonra hangi duruma geçer? Geçişleri diyagramda gösterin.
4. `/home/ali` klasöründeyken `../ayse/notlar.txt` görelî yolunun mutlak karşılığını yazın.
5. Windows ve Linux gereken bir test ortamı için sanal makine mi kapsayıcı mı seçersiniz? Nedenini çekirdek paylaşımıyla açıklayın.
6. Bir uygulamanın dosyaya yazmasını; sistem çağrısı, çekirdek modu, dosya sistemi ve disk sürücüsü adımlarıyla sırayla anlatın.

Kaynaklar

- Silberschatz, Galvin ve Gagne, Operating System Concepts.
- Tanenbaum ve Bos, Modern Operating Systems.
- The Linux Foundation ve Free Software Foundation lisans kaynakları.

Son

Yazılım ve İşletim Sistemleri

Sonraki sunum: [Windows İşletim Sistemi](#) · [Çalışma soruları](#) · Simülatör: [Süreç Zamanlama](#) · [Terim sözlüğü](#)